

Solving response of tuned mass damper with ODE45

+ Follow

81 views (last 30 days)

Show older comments



Frederik Rolighed Christensen
on 17 May 2022

👍 Vote | 0 🔗 Link 🚩 Flag

Edited: Frederik Rolighed Christensen on 18 May 2022

Hello there

I am currently trying to model a 2 DOF tuned mass damper system. The system consist of two masses, m_1 and m_2 , connected in series by two springs, k_1 and k_2 (see below). In order for the Tuned Mass Damper to work, the natural frequency needs to be idental for both sub-systems, that is $\omega = \sqrt{k_1/m_1} = \sqrt{k_2/m_2}$. If m_1 is a added a forced vibration of $F(t)$ with frequency of ω , the expected response is that m_1 should be static, while m_2 should oscillate a lot.

My problem is that I do not see this happening even though i analysed both systems individually beforehand. I believe i inserted the time-dependent variable correctly by use of `interp1`, but the results are not convincing. Does anyone see my error?

Furthermore, is it not possible to set initial conditions to zero for all conditions, when the system is forced to vibrate by F ?

```
clc; clear all; close all;
```

```
%
```

```
%      |                               |---> x1                               |---> x2
%      |                               |                               |
%      |      k1      |      k2      |
%      |____/\  /\  ____| m1 |____/\  /\  ____| m2 |
%      |      \  \  \  |____|      \  \  \  |____|
%      |
%      |
%      |
```

```
%
```

MATLAB Answers

```
tspan = linspace(0, 500, 5000);
```

```
F0 = 10;
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 1 DOF sub-systems %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%% System 1 1DOF:
```

```
% Mass and stiffness:
```

```
m1 = 80;
```

```
k1 = 160;
```

```
% Driving force
```

```
t_F = tspan;
```

```
omega = sqrt(k1/m1);
```

```
F = F0.*sin(omega.*t_F);
```

```
y0 = [0 1];
```

```
[t1,y1] = ode45(@(t,y) odefun_test_1(t, y, m1, k1, t_F, F), tspan, y0);
```

```
%% System 2 1DOF:
```

```
m2 = 10;
```

```
k2 = 20;
```

```
% Driving force
```

```
t_F = tspan;
```

```
omega = sqrt(k2/m2);
```

```
F = F0.*sin(omega.*t_F);
```

```
y0 = [0 1];
```

```
[t2,y2] = ode45(@(t,y) odefun_test_1(t, y, m2, k2, t_F, F), tspan, y0);
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 2 DOF systems %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
% Driving force
```

```
t_F = linspace(0, 500, 5000);
```

```
omega = sqrt(k1/m1);
```

```
F = F0.*sin(omega.*t_F);
```

```
y0 = [0 1 0 0];
```

```
[t3,y3] = ode45(@(t,y) odefun_test_2(t, y, m1, k1, m2, k2, t_F, F), tspan, y0);
```

```
M = [m1 0; 0 m2];
```

```
K = [(k1+k2) -k2; -k2 k2];
```

```
A = inv(M)*K;
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Plot %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
figure()
```

```
subplot(1,2,1)
```

```
plot(t1, y1(:,1));title('System 1: x')
```

```
subplot(1,2,2)
```

```
plot(t1, y1(:,2));title('System 1: xd')
```

```
figure()
```

```
subplot(1,2,1)
```

```
plot(t2, y2(:,1));title('System 2: x')
```

```
subplot(1,2,2)
```

```
plot(t2, y2(:,2));title('System 2: xd')
```

```
figure()
```

```
subplot(2,2,1)
```

```
plot(t3, y3(:,1)+y3(:,3));title('System 1/2: x_2')
```

```
subplot(2,2,2)
```

```
plot(t3, y3(:,2)+y3(:,4));title('System 1/2: xd_2')
```

```
subplot(2,2,3)
```

```
plot(t3, y3(:,1));title('System 1/2: x_1')
```

```
subplot(2,2,4)
```

```
plot(t3, y3(:,2));title('System 1/2: xd_1')
```

```

%% Eigenfrequencies
% Eigenfrequencies

```

```

disp('-----')

```

```

% System 1:

```

```

disp('System 1: [Hz]')

```

```

omega_anal1 = sqrt(k1/m1)/(2*pi)

```

```

disp('-----')

```

```

% System 1:

```

```

disp('System 2: [Hz]')

```

```

omega_anal2 = sqrt(k2/m2)/(2*pi)

```

```

disp('-----')

```

```

% System 3:

```

```

disp('System 3: [Hz]')

```

```

omega_anal3 = sqrt(eig(A))/(2*pi)

```

```

disp('-----')

```

```

%% Functions
% Functions

```

```

%% System (1 DOF):

```

```

function dfdt_1 = odefun_test_1(t, y, m, k, t_F, F)

```

```

    dfdt_1 = zeros(2,1);

```

```

    F = interp1(t_F, F, t);

```

```

    % State variables:

```

```

    x = y(1);

```

```

    xd = y(2);

```

```

    % State equations:

```

```

    dfdt_1(1) = xd;

```

```

    dfdt_1(2) = -k*x/m + F./m;

```

```

end

```

```

%% System (2 DOF):

```

```

function dfdt_3 = odefun_test_2(t, y, m1, k1, m2, k2, t_F, F)

```

```

    dfdt_3 = zeros(4,1);

```

```

    F = interp1(t_F, F, t);

```

```

    % State variables:

```

```

    x1 = y(1);

```

```

    xd1 = y(2);

```

```

    x2 = y(3);

```

```

    xd2 = y(4);

```

```

    % State equations:

```

```

dfdt_3(1) = xd1;
dfdt_3(2) = -(k1+k2)/m1*x1 + k2/m1*x2 + F./m1;
dfdt_3(3) = xd2;
dfdt_3(4) = k2/m2*x1 - k2/m2*x2;
end

```

end

 0 Comments



Comment on this question...

Answer this question

Answers (1)



Sam Chak **LEVEL 7** **EDITOR**

on 17 May 2022

Edited: Sam Chak **LEVEL 7** **EDITOR** on 17 May 2022

 Vote | 1  Link  Flag

Hi @Frederik Rolighed Christensen

Please note that your Tuned Mass–Damper System does not contain any damper. Also check if the results are expected. Your thinking is correct, but you applied the sinusoidal force at the wrong phase. As an alternative, I have added an option for you to change the original force vector **F** to computed force inputs **Fc**, **Fu**, or **F1**. Given the system:

```

function dydt = odefcn(t, y)

    % Parameters
    m1 = 80;
    m2 = 10;
    k1 = 160;
    k2 = 20;
    f = 10;
    omega = sqrt(k1/m1);

    % Mass matrix
    M = [m1 0; 0 m2];

    % Stiffness matrix
    K = [k1+k2 -k2; -k2 k2];

    % State Matrix
    A = [zeros(length(M)) eye(length(M)); -M\K zeros(length(M))];

    % Input Matrix
    B = [zeros(2, 2); M\eye(2)];

    % Force Vector
    F = [f*sin(omega*t); 0]; % original force F(t) applied to mass m1
    Fc = [-2*f*cos(omega*t); 0]; % Corrected Force by Sam (similar to F1 below)

    % Required Force to achieve desired System Performance dydt = - H*y
    Q = 1/3;
    H = [zeros(length(M)) eye(length(M)); (Q^2)*eye(2) 2*Q*eye(2)];
    G = B\ (H + A);
    Fu = - G*y;
    F1 = Fu.*[1; 0];

    % State-space model
    dydt = A*y + B*F; % Change F to Fu or F1 to see other results

end

```

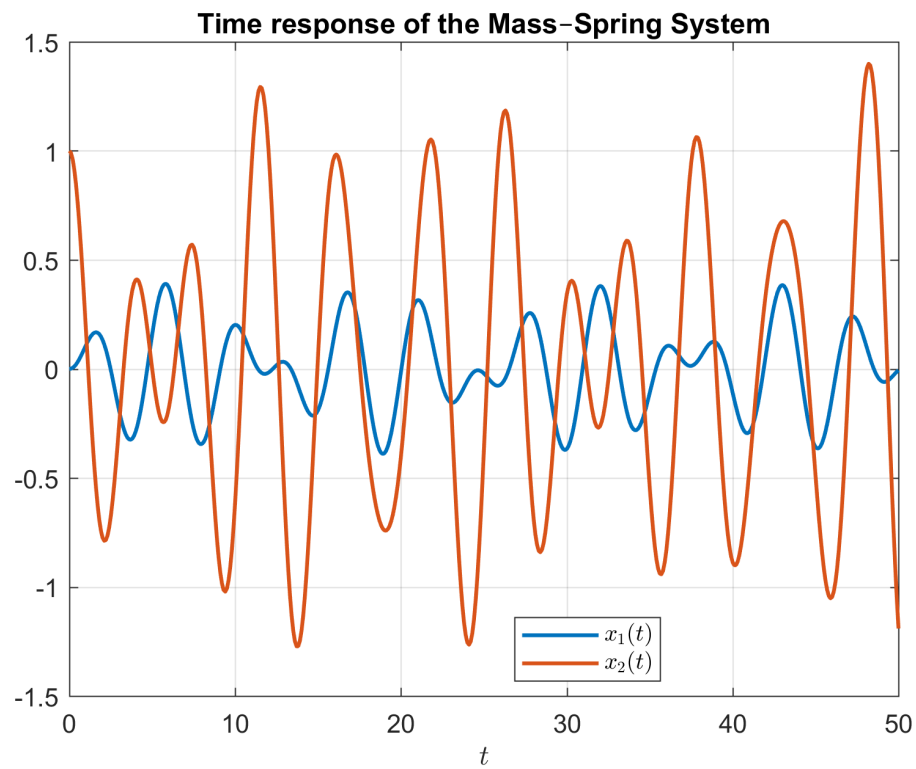
and run the ode45 solver

```

tspan = 0:0.1:50;
y0 = [0 1 0 0];
[t, y] = ode45(@odefcn, tspan, y0);

```

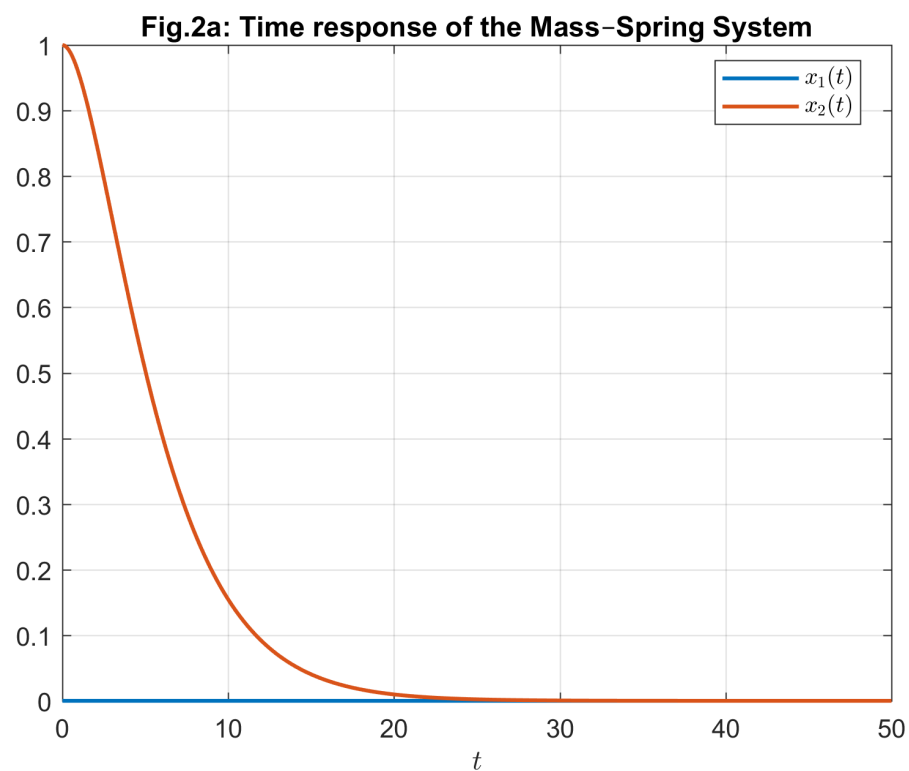
to obtain

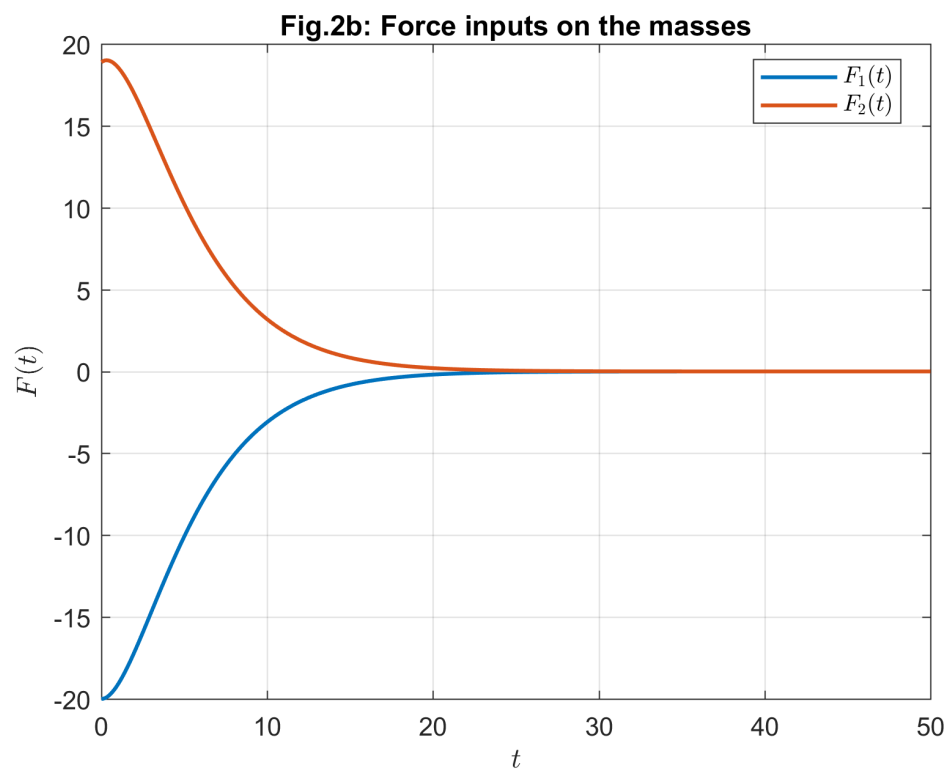


If $\mathbf{F}$ is changed to $\mathbf{F}_u$, then

```
dydt = A*y + B*Fu;
```

Theme Copy





Note that the force $F_1(t)$ still needs to be applied to the mass m_1 to compensate for the effect of $m_2\ddot{x}_2$ in order to maintain it at the equilibrium, $x_1 = 0$.

If you want m_1 to remain stationary but m_2 oscillates, then F should be changed to F1.

```
dydt = A*y + B*F1;
```

Theme Copy

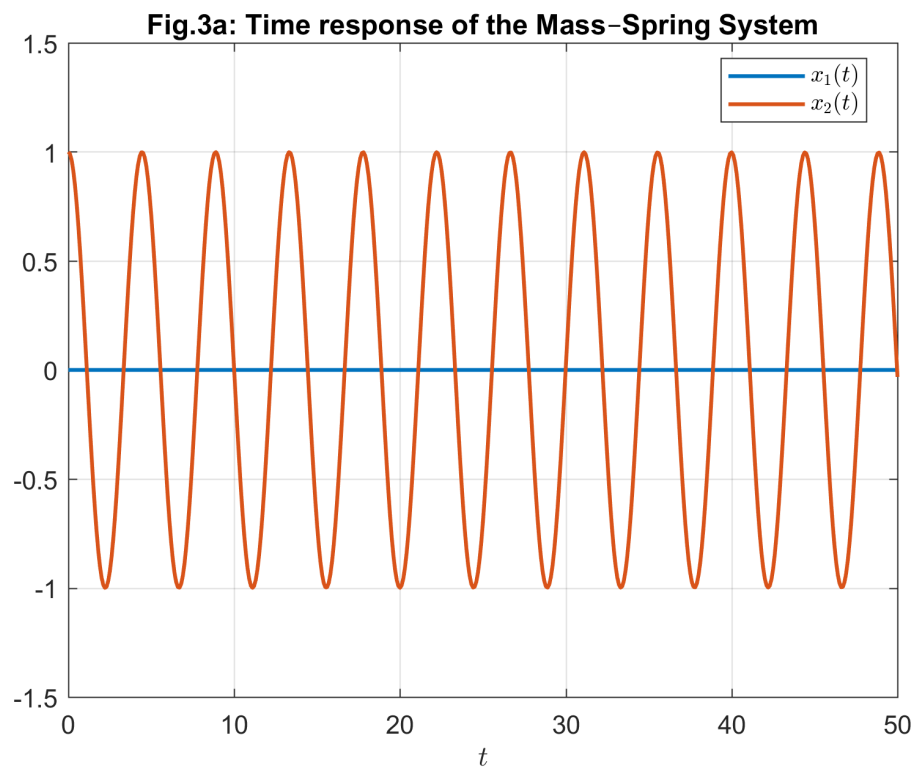
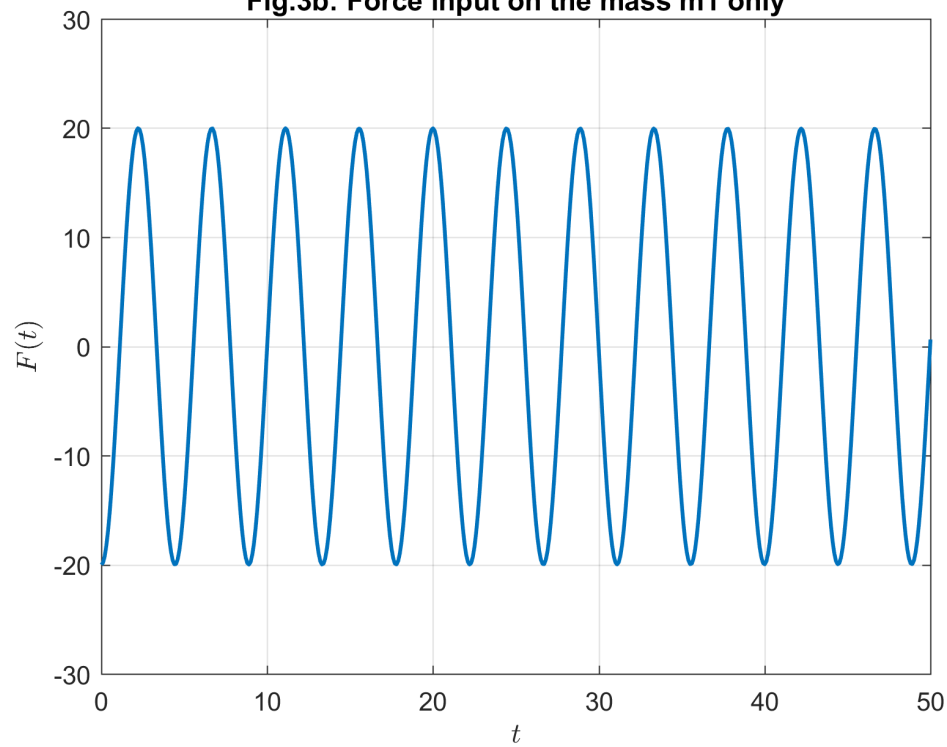


Fig.3b: Force input on the mass m1 only



3 Comments [Show 1 older comment](#)



Sam Chak **LEVEL 7 EDITOR** on 18 May 2022



Edited: Sam Chak **LEVEL 7 EDITOR** on 18 May 2022

Hi @Frederik Rolighed Christensen

Thanks for your reply. The mass matrix M and stiffness matrix K for the TMD actually came from your own model (see image below). I just expressed them properly in the *Matrix form* called **State-space model**

$$\dot{\mathbf{y}} = \mathbf{A}\mathbf{y} + \mathbf{B}\mathbf{F}$$

so that **ODE45** solver can integrate the model.

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 2 DOF systems %%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Driving force
t_F = linspace(0, 500, 5000);
omega = sqrt(k1/m1);
F = F0.*sin(omega.*t_F);

y0 = [0 1 0 0];
[t3,y3] = ode45(@(t,y) odefun_test_2(t, y, m1, k1,

M = [m1 0; 0 m2];
K = [(k1+k2) -k2; -k2 k2];
A = inv(M)*K;
```

In order to stabilize the mass m_1 subject to the oscillating (disturbing) mass m_2 ,

your concept of applying the sinusoidal force with frequency $\sqrt{\frac{k_1}{m_1}}$ at mass

m_1 to compensate for the disturbance effect of m_2 is **CORRECT!**

However, the sinusoidal force should be in this form:

$F_1 = f \sin(\omega t + \phi)$, where ϕ is the phase. Since the initial condition $x_2(0) \neq 0$ and $\dot{x}_2(0) = 0$, we can intellectually guesstimate that $x_2(t)$ begins to oscillate at the positive peak value.

So, in order to compensate for this positivity effect, we add the phase in the negative

direction $\phi = -90^\circ = -\frac{\pi}{2} \text{ rad}$, and we have

$$F_1 = f_1 \sin\left(\omega t - \frac{\pi}{2}\right) = -f_1 \cos(\omega t), \text{ which I}$$

inserted the corrected force vector, **Fc**. Search for "Fc", or edit this line:

```
phi = - pi/2;  
F = 2*F0*sin(omega*t_F + phi);
```

Theme Copy

----- **EXTRA** -----

If your project does not require you to stabilize both m_1 and m_2 , then you can ignore my suggested force inputs, **Fu** that is created using **Q**, **H**, **G**. For your info, **Q** is the desired system's undamped frequency, **H** is the desired stable matrix, such that the TMD behaves

$$\dot{\mathbf{y}} = -\mathbf{H}\mathbf{y}$$

(that is equivalent to desired mass-damper-spring response:

$$\ddot{\mathbf{x}} = -\mathbf{Q}^2\mathbf{I}\mathbf{x} - 2\mathbf{Q}\mathbf{I}\dot{\mathbf{x}})$$

after the suggested force vector

$$\mathbf{F}_u = -\mathbf{G}\mathbf{y}$$

is injected into the TMD and becomes

$$\dot{\mathbf{y}} = \mathbf{A}\mathbf{y} + \mathbf{B}(-\mathbf{G}\mathbf{y})$$

$$\dot{\mathbf{y}} = (\mathbf{A} - \mathbf{B}\mathbf{G}) \mathbf{y}$$

Equating desired $-\mathbf{H}$ to $\mathbf{A} - \mathbf{B}\mathbf{G}$ and solve for **G** leads to

$$-\mathbf{H} = \mathbf{A} - \mathbf{B}\mathbf{G}$$

$$\mathbf{G} = \mathbf{B}^{-1}(\mathbf{H} + \mathbf{A})$$

Hello Sam!

Thanks for your elaborate answer! By adding a little damping to get rid of oscillations due to initial conditions, I have now a model of the tuned mass damper. The code is below and shows that for $t \rightarrow \infty$, the oscillations for mass 1 becomes close to zero (not totally zero due to the damping coefficient) and the oscillations for mass 2 are constant and much larger than for mass 1.

Theme Copy

```
clc; clear all; close all;

% TMD test script
%
%      |                               |---> x1                               |---> x2
%      |
%      |_____/\  /\  _____| m1 |_____/\  /\  _____| m2 |
%      |          \/\  \/\  |_____|          \/\  \/\  |_____|
%      |
%      |
%
tspan = 0:0.1:1000;
F0 = 10;

%% System 1 1DOF:
% Mass and stiffness:
m1 = 80;
k1 = 160;

% Driving force
t_F = tspan;
omega = sqrt(k1/m1);
F = F0.*sin(omega.*t_F);

y0 = [0 1];
[t1,y1] = ode45(@(t,y) odefun_test_1(t, y, m1, k1, t_F, F), tspan,

%% System 2 1DOF:
m2 = 10;
k2 = 20;

% Driving force
t_F = tspan;
omega = sqrt(k2/m2);
F = F0.*sin(omega.*t_F);

y0 = [0 1];
[t2,y2] = ode45(@(t,y) odefun_test_1(t, y, m2, k2, t_F, F), tspan,

%% System 1/2 2DOF (with minor damping):
% Driving force
t_F = tspan;
omega = sqrt(k1/m1);
phi = 0;
F = F0.*sin(omega.*t_F + phi);
```

```

c2 = 0.1;

y0 = [0 1e-5 0 0];
[t3,y3] = ode45(@(t,y) odefun_test_2(t, y, m1, k1, m2, k2, c2, t_F,

M = [m1 0; 0 m2];
K = [(k1+k2) -k2; -k2 k2];
A = M\K;

%% Plot

figure()
subplot(1,2,1)
plot(t1, y1(:,1));title('System 1: x')
subplot(1,2,2)
plot(t1, y1(:,2));title('System 1: xd')

figure()
subplot(1,2,1)
plot(t2, y2(:,1));title('System 2: x')
subplot(1,2,2)
plot(t2, y2(:,2));title('System 2: xd')

figure()
plot(t3, y3(:,1), 'linewidth', 2);hold on;
plot(t3, y3(:,1)+y3(:,3), 'linewidth', 2);hold off
set(gca, 'fontsize', 18);grid on
legend('x_1(t)', 'x_2(t)')

figure()
plot(t3, F, 'linewidth', 2);grid on
legend('F(t)')
set(gca, 'fontsize', 18)

%% Eigenfrequencies:
disp('-----')

% System 1:
disp('System 1: [Hz]')
plotOn = 0;
fft_input = y1(:,1);
[omega_fft_, omega_tangent_] = natFreq(fft_input, t1, plotOn)
omega_anal1 = sqrt(k1/m1)/(2*pi)
disp('-----')

% System 1:
disp('System 2: [Hz]')
fft_input = y2(:,1);
[omega_fft_, omega_tangent_] = natFreq(fft_input, t2, plotOn)
omega_anal2 = sqrt(k2/m2)/(2*pi)
disp('-----')

% System 3:
disp('System 3: [Hz]')

```

```

plotOn = 1;
fft_input = y3(:,1)+y3(:,3);
[omega_fft_, omega_tangent_] = natFreq(fft_input, t3, plotOn)
omega_anal3 = sqrt(eig(A))/(2*pi)
disp('-----')

%% System (1 DOF):
function dfdt_1 = odefun_test_1(t, y, m, k, t_F, F)
    dfdt_1 = zeros(2,1);

    F = interp1(t_F, F, t);

    % State variables:
    x = y(1);
    xd = y(2);

    % State equations:
    dfdt_1(1) = xd;
    dfdt_1(2) = -k*x/m + F./m;
end

%% System (2 DOF):
function dfdt_3 = odefun_test_2(t, y, m1, k1, m2, k2, c2, t_F, F)

    dfdt_3 = zeros(4,1);
    F = interp1(t_F, F, t);

    % State variables:
    x1 = y(1);
    xd1 = y(2);
    x2 = y(3);
    xd2 = y(4);

    % State equations:
    dfdt_3(1) = xd1;
    dfdt_3(2) = -(k1+k2)/m1*x1 + k2/m1*x2 + F./m1;
    dfdt_3(3) = xd2;
    dfdt_3(4) = k2/m2*x1 - k2/m2*x2 - c2/m2*xd2;
end

```



Comment on this answer...

Answer this question

Normal ▼

TEXT

CODE

INSERT

RUN

HELP

☒ Follow activity on this question (change notification settings)

Cancel

Submit

Remember to follow the Community Guidelines

See Also

MATLAB Answers

unrecognized function or variable for spring mass damper with forcing function

1 Answer

value assigned to variable might be unsure (Ln 12)

1 Answer

2 Degree of Freedom system with ODE 45

1 Answer

Entire Website

lagrange_eq(T,V,Q,F0,r_F,fun_name,varargin)

File Exchange

filter4fun

File Exchange

ball in a cone - Lagrange mechanics

File Exchange

Categories

Signal Processing > Signal Processing Toolbox > Vibration Analysis

Find more on **Vibration Analysis** in Help Center and File Exchange

Tags

tuned mass damper

ode45

time

dependent

mechanical

vibration

oscillating

spring

mass

dof

degree of freedom

vibration absorber

Products

MATLAB

Release

R2021a

Poll

Today, while using MATLAB, I'm going to....

- ☐ figure out what my old code does
- ☐ write code comments for future me
- ☐ address a code analyzer warning
- ☐ reflect on the skills I've gained
- ☐ get food on my mouse and keyboard
- ☐ All of the above

11819 votes 10 comments 406 views

Vote

mathworks.com

© 1994-2024 The MathWorks, Inc. MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See [mathworks.com/trademarks](https://www.mathworks.com/trademarks) for a list of additional trademarks. Other product or brand names may be trademarks or registered trademarks of their respective holders.

Join the conversation